

What does all that code do?

Optional · not graded · about 15 minutes per worksheet

In HW3, an AI can write every code cell for you in seconds. These worksheets help you understand what those cells do. Each one goes with one HW3 question. Keep your own HW3 notebook open beside it, and do as many as you like, in any order.

No coding background? Start with the HW3 warm-up notebook. It introduces the Python you need, one small idea at a time, with self-checking exercises.

START HERE

Four moves, every time

1 · PREDICT

Before running, write down what you expect to see.

2 · TRACE

Work through the calculation by hand, using tiny numbers.

3 · TWEAK

Change one thing, predict again, then run.

4 · EXPLAIN

Close the AI chat. Explain the idea in two sentences of your own.

A wrong prediction is not a failure. It shows you exactly which line of code to look at again.

HW3 QUESTION	WORKSHEET	WARM-UP NOTEBOOK
Q1.1 · 1.1 Load and split	01	Sections 7–8
Q1.1 · 1.2 and 1.4 Baseline, RMSE, R^2	02	Section 4
Q1.1 · 1.3 Visualization	03	Section 9
Q1.2 Overfitting demo	04	Section 11
Q1.3 Decision boundary	05	Sections 3, 5
Bonus Q1.4 Gradient descent	06	Section 10
Part 2 · Chapter 4	07	Section 6

Load and split the data

≈ 10 minutes

PREDICT · BEFORE YOU PROMPT

1. In the California Housing dataset, what does **one row** describe? What is the **target**, the value we try to predict? (You can look this up.)

2. The dataset has 20,640 rows. With a 60/20/20 split, how many rows go to each part?

train _____ validation _____ test _____

TRACE · AFTER IT RUNS

3. Find the line or lines in your code that make the split. Copy the line that sets the sizes:

4. Many solutions call `train_test_split` twice: first with `test_size=0.4`, then with `test_size=0.5`. Why does 0.5 in the second call produce 20% of the whole dataset?

TWEAK

5. Change `random_state=42` to another number and rerun. Before you run: will the *sizes* change? Will the *first five rows of the training set* change?

EXPLAIN

6. Why do we leave the test set alone until the very end?

Baseline, RMSE and R^2 , by hand

≈ 15 minutes · calculator welcome

Four houses, with values in units of \$100,000. The **baseline** always guesses the mean, which is 3. The **model** makes its own guesses. For each guess, the error is $\text{guess} - \text{actual}$.

TRACE

house	actual	baseline guess	baseline error	error ²	model guess	model error	error ²
A	1	3	2	4	1.5	0.5	0.25
B	2	3			2		
C	3	3			3.5		
D	6	3			5		
mean of error ² (MSE)					mean of error ² (MSE)		
RMSE = $\sqrt{\text{MSE}}$					RMSE = $\sqrt{\text{MSE}}$		

$$R^2 = 1 - (\text{model MSE} \div \text{baseline MSE}) = \underline{\hspace{2cm}}$$

TRACE · IN YOUR CODE

Find the line in your notebook that computes RMSE. Copy it, and label the part that does each step: **error**, **square**, **mean**, **root**. (Sometimes a library function does two steps at once.)

EXPLAIN

What does the R^2 value in *your* notebook say about your linear model compared with always guessing the mean? Which is better, and by how much?

Match the picture to the code

≈ 10 minutes

Every visual element in your plot comes from a specific line of code. For each element, find its line in your notebook and write down the key part, such as `--` or `facecolors="none"`.

In the picture	The code that draws it
dashed regression line	
solid observed points	
hollow predicted points	
dotted residual lines	
axis limits	
choosing about 12 evenly spaced points	

TWEAK

Predict first, then try each change:

1. Change the 12 points to 4. What will the plot look like?
2. Make the predicted points filled instead of hollow. Why might hollow be the better choice?

EXPLAIN

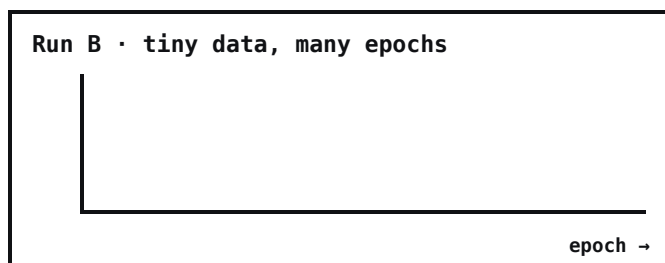
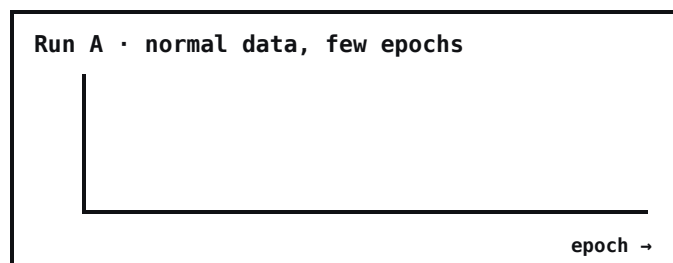
What does a *long* residual line mean? What does it mean when the observed point sits *above* the line?

Read the overfitting curves

≈ 15 minutes

PREDICT · SKETCH BEFORE YOU RUN

Draw what you expect the training loss and validation loss to look like in each run. Use two different pens or line styles.



TRACE · IN YOUR CODE

Find the two settings that make Run B different from Run A. Write down their variable names and values.

training-set size: _____ number of epochs: _____

Find the loop that trains the model one epoch at a time and records the losses. Which line records the training loss, and which records the validation loss?

COMPARE

How did your sketch compare with the real plot? At which epoch would you stop Run B, and why?

TWEAK

Give Run B ten times more training examples. Predict the new curves, then run. Did the gap between the two curves grow or shrink?

EXPLAIN

Finish this sentence using what your plot shows: "Overfitting is when..."

WORKSHEET 05 · Q1.3

A decision boundary, by hand

≈ 15 minutes

A classifier computes a **score** = (each feature × its weight, all added up) + bias. A positive score puts a point on one side of the boundary; a negative score puts it on the other. The boundary itself is wherever the score is exactly 0.

Suppose the quadratic model uses features $[x_1, x_2, x_1^2, x_1 \cdot x_2, x_2^2]$, weights $[0, 0, -1, 0, -1]$ and bias 0.45 . Most of the weights are 0, so the score simplifies to $0.45 - x_1^2 - x_2^2$. (Your notebook may use different numbers; you can redo this table with yours.)

TRACE

point (x_1, x_2)	x_1^2	x_2^2	score = $0.45 - x_1^2 - x_2^2$	+ or -?
A (0.2, 0.1)	0.04	0.01	0.40	+
B (0.9, 0.3)				
C (0.0, 0.7)				

Now use the **linear** model, with features $[x_1, x_2]$, weights $[1, 0]$ and bias 0 . What is its score for A, B and C? What shape is its boundary?

THINK

The quadratic boundary is where $x_1^2 + x_2^2 = 0.45$. That is a circle. What is its radius? Which bias would make a circle of radius 0.5?

Why can no straight line separate one ring of points from another ring around it?

TWEAK

In your notebook, change `BIAS_QUADRATIC`. Predict first: does the circle grow or shrink?

WORKSHEET 06 · BONUS Q1.4

One step of gradient descent

≈ 15 minutes · calculator welcome

The model predicts $w \times x$. The data are $x = [1, 2, 3]$ and $y = [2, 4, 6]$, so the best w is 2. Start at $w = 0$ with a learning rate of 0.1.

```
errors    = w * x - y
loss       = mean(errors2)
gradient  = 2 * mean(errors * x)
new w     = w - learning_rate * gradient
```

TRACE

step	w	predictions $w \cdot x$	errors	loss	errors $\cdot x$	gradient	new w
0	0	0, 0, 0	-2, -4, -6				
1							

THINK

The first gradient is negative. Why does that make w go *up*?

Redo step 0 with a learning rate of 0.25. Is the new w closer to 2 or farther away than 0 was? What do you think happens if you keep going?

CHECK IN CODE

Section 10 of the warm-up notebook runs this exact loop. Compare its printout with your table, then try learning rates of 0.2 and 0.25.

WORKSHEET 07 · PART 2

Reading dense textbook code

≈ 15 minutes

Chapter 4 packs a lot into each line. Two habits make it readable: **unchain** a line into one step per line, and **rewrite** library calls as plain loops. Practice both here on lines that HW3 does not grade, then use them on the Q2 questions.

UNCHAIN

Early in the chapter, the book compares one image of a 3 with the "average 3":

```
dist_3_abs = (a_3 - mean3).abs().mean()  
dist_3_sqr = ((a_3 - mean3)**2).mean().sqrt()
```

Pretend each image has only three pixels: `a_3 = [0, 0.5, 1]` and `mean3 = [0.2, 0.5, 0.6]`. Fill in one row per step.

step	code	what it does	result
1	<code>a_3 - mean3</code>		
2a	<code>.abs()</code>		
3a	<code>.mean()</code>		
2b	<code>**2</code>		
3b	<code>.mean()</code>		
4b	<code>.sqrt()</code>		

You have seen the second line before, under a different name. Which worksheet was it in?

REWRITE AS A LOOP

Chapter 4 often uses `torch.where(condition, a, b)`. Item by item, it means "if the condition holds, take `a`; otherwise take `b`". Here is a practice line (not from the book) and its plain-Python version:

```
labels = torch.where(temps > 20, 1, 0)

# plain Python
labels = []
for t in temps:
    if t > 20:
        labels.append(1)
    else:
        labels.append(0)
```

Now write the plain-Python version of `(temps > 20).float().mean()`. What does the result tell you about `temps`?

CONNECT

Chapter 4 lists seven steps for training a model (initialize, predict, loss, gradient, step, repeat, stop). Which lines of the Worksheet 06 loop match each step?

ANY QUESTION

Using AI beyond "write it for me"

The AI that wrote your code can also help you understand it. Paste your cell and try one of these prompts:

PREDICT WITH ME

"Explain this cell line by line for a beginner. After each line, ask me to predict what it does before you tell me."

QUIZ ME

"Ask me three short questions about this code. Wait for my answers before giving yours."

SHOW EVERY STEP

"Rewrite this without numpy or sklearn, as plain Python loops, so I can see every step."

WHAT MATTERS

"Which single line matters most here? What would happen if I deleted it? Let me guess first."

FIND THE BUG

"Put one small bug in this code and let me find it."

TINY NUMBERS

"Run this idea on a dataset of 4 rows so I can check every number by hand."

THE EXPLAIN-BACK TEST

Close the chat. Write two sentences about what your cell does. Then paste them back and ask, "What did I get wrong or leave out?" If you can explain it without the chat open, you understand it.

Next week, HW4 asks you to add comments to a code file, trace a tiny model by hand, and write a prompt that describes code. Those are the same moves you practiced here.